Java 入门笔记

第七课 · Java 一维数组

学习目标	掌握一维数组的声明、创建、初始化、下标访问和遍历，并理解数组引用、默认值与越界问题。

目录

本课目标 2

4.1 为什么需要数组 2

4.2 声明与创建 2

4.3 数组初始化 3

4.4 数组元素与 length 3

4.5 数组的引用与内存关系 4

4.6 遍历数组 4

4.7 综合示例：成绩统计 5

4.8 常用工具方法 5

常见错误 5

本课小结 5

自测题 6

整理日期：2026 年 11 月 1 日

> 第六课解决“重复处理”的问题；数组解决“批量保存同类型数据”的问题。循环与数组结合后，可以统一处理几十个甚至更多数据。

本课目标

- 理解数组的用途、特点与引用含义。
- 掌握一维数组的声明、创建和初始化。
- 会使用下标、length、普通 for 和增强 for。
- 理解默认值、数组越界和基本内存关系。

4.1 为什么需要数组

如果要保存 50 名学生的成绩，定义 50 个变量难以统一处理。数组使用一个名字管理一组同类型数据，并通过下标访问其中的元素。

```
double[] scores = new double[50];
```

Java 数组具有以下特点：

- 数组是对象，数组变量保存对数组对象的引用。
- 同一个数组中的元素类型相同。
- 创建后长度固定，不能直接扩展或缩短。
- 下标从 0 开始，最后一个下标是 length - 1。

4.2 声明与创建

4.2.1 声明数组变量

```
int[] numbers;  // 推荐：类型信息集中在一起
int numbers2[]; // 合法，但可读性较弱
```

声明数组变量只创建了引用变量，没有创建数组对象，也不能在声明的方括号中填写长度。

```
int[5] numbers; // 错误
```

4.2.2 使用 new 创建数组

```
int[] numbers;
numbers = new int[5];
```

也可以合并为一条语句：

```
int[] numbers = new int[5];
```

new int[5] 在运行时创建一个包含 5 个 int 元素的数组对象。

4.3 数组初始化

4.3.1 动态初始化

动态初始化只指定长度，元素由系统赋予默认值。

```
int[] ages = new int[3];
String[] names = new String[3];
```

元素类型	默认值
byte、short、int、long	0
float、double	0.0
char	\u0000
boolean	false
引用类型	null

局部变量本身没有自动默认值，但 new 创建的数组元素有默认值。

4.3.2 静态初始化

静态初始化直接给出元素，数组长度由元素个数决定。

```
int[] numbers = {10, 20, 30};
String[] names = {"张三", "李四", "王五"};
```

完整写法：

```
int[] numbers = new int[]{10, 20, 30};
```

使用 new int[]{...} 时不能再在方括号中指定长度。

4.4 数组元素与 length

```
int[] numbers = {10, 20, 30};
System.out.println(numbers[0]);              // 10
System.out.println(numbers[2]);              // 30
System.out.println(numbers.length);          // 3
System.out.println(numbers[numbers.length - 1]); // 30
```

数组长度使用属性 length，没有圆括号。字符串长度则使用方法 length()，两者不要混淆。

```
numbers[0] = 100;
System.out.println(numbers[0]); // 100
```

合法下标范围为 0 到 length - 1。访问 numbers[numbers.length] 会在运行时抛出 ArrayIndexOutOfBoundsException。

4.5 数组的引用与内存关系

```
int[] first = {10, 20, 30};
int[] second = first;
second[0] = 99;
System.out.println(first[0]); // 99
```

first 和 second 保存了同一个数组对象的引用，因此通过任一变量修改元素，另一变量访问时也能看到变化。



直接输出数组变量通常得到类似 [I@1b6d3586 的默认字符串表示，它不是可供使用的内存地址。要输出元素，应遍历数组或使用 Arrays.toString。

4.6 遍历数组

4.6.1 普通 for 循环

普通 for 能获得下标，既可读取元素，也可修改元素。

```
int[] numbers = {10, 20, 30, 40, 50};
int sum = 0;
for (int i = 0; i < numbers.length; i++) {
    sum += numbers[i];
}
System.out.println("总和：" + sum);
```

4.6.2 增强 for 循环

增强 for 适合从头到尾读取所有元素。

```
for (int number : numbers) {
    System.out.println(number);
}
```

增强 for 中的 number 是当前元素值的临时变量。给它重新赋值不会修改基本类型数组中的原元素。

```
for (int number : numbers) {
    number = 0; // 不会把数组元素改成 0
}
```

需求	推荐方式
需要下标或修改元素	普通 for
只需依次读取元素	增强 for

4.7 综合示例：成绩统计

```
public class ScoreStatistics {
    public static void main(String[] args) {
        double[] scores = {86.5, 92.0, 78.5, 88.0, 95.5};
        double sum = 0;
        double max = scores[0];

        for (double score : scores) {
            sum += score;
            if (score > max) {
                max = score;
            }
        }

        double average = sum / scores.length;
        System.out.printf("平均分：%.2f%n", average);
        System.out.println("最高分：" + max);
    }
}
```

如果数组可能为空，就不能直接读取 scores[0]，计算平均值时也要避免除以 0。

4.8 常用工具方法

java.util.Arrays 提供了常见数组工具。

```
import java.util.Arrays;

int[] numbers = {30, 10, 20};
System.out.println(Arrays.toString(numbers)); // [30, 10, 20]
Arrays.sort(numbers);
System.out.println(Arrays.toString(numbers)); // [10, 20, 30]
```

常见错误

- 把 double[] scores = double[50]; 写成缺少 new 的形式。
- 声明数组时写 int[5] numbers。
- 循环条件写成 i <= numbers.length，导致最后一次越界。
- 把数组的 length 写成 length()。
- 认为给增强 for 的基本类型变量赋值会修改数组元素。
- 认为直接打印数组变量会自动列出所有元素。

本课小结

- 数组是长度固定、元素类型相同的对象。
- 数组变量保存引用；声明变量与创建数组是两个动作。
- 下标范围是 0 到 length - 1。
- 普通 for 适合下标和修改，增强 for 适合依次读取。

- 二维数组本质上是“元素仍然是数组”的数组，这是第八课的主题。

自测题

1. int[] a; 是否已经创建数组对象？
2. new boolean[3] 中三个元素的默认值是什么？
3. 为什么数组最后一个元素的下标是 length - 1？
4. 编写程序：计算整数数组中的最小值及其第一次出现的下标。